

Aprendizado Auto-Supervisionado de Espaços Latentes com Poucas Amostras: Impacto da Aumentação de Dados por Difusão e Validação via Agrupamento

Carlos E. Cavalieri Furtado¹

¹Instituto de Computação – Universidade Estadual de Campinas (UNICAMP)

Resumo. *Este trabalho investiga o impacto da aumento de dados baseada em modelos de difusão no aprendizado auto-supervisionado de espaços latentes em cenários com poucas amostras. Utilizando o dataset Corel com seis classes e aproximadamente 60 imagens por classe, implementamos e comparamos três técnicas de extração de características: BYOL (Bootstrap Your Own Latent), CNN-JEPA (Joint Embedding Predictive Architecture) e DGAE (Diffusion-Guided Autoencoder). Avaliamos o desempenho de agrupamento em três cenários: (i) dataset original, (ii) aumentado com Stable Diffusion via fine-tuning LoRA, e (iii) aumentado com modelo de difusão treinado do zero. Os resultados são analisados através de métricas de agrupamento (Silhouette Score, Adjusted Rand Index e Normalized Mutual Information) e visualizações UMAP, fornecendo insights sobre a eficácia de diferentes estratégias de aumento sintético para aprendizado auto-supervisionado com poucos dados.*

1. Introdução

O aprendizado auto-supervisionado atualmente é uma das áreas mais promissoras do aprendizado de máquina, permitindo que modelos aprendam representações ricas sem necessidade de grandes volumes de dados rotulados [Jing and Tian 2020, LeCun and Misra 2021]. Técnicas como SimCLR [Chen et al. 2020], BYOL [Grill et al. 2020], DINO [Caron et al. 2021] e suas variantes demonstraram capacidade de extrair características visuais de alta qualidade através de objetivos de aprendizado baseados em contrastes ou predições, utilizando diferentes aumentações da mesma imagem para treinamento.

No entanto, um desafio fundamental dessas abordagens reside na qualidade e diversidade das aumentações de dados. Métodos tradicionais de aumento, como color jittering, rotações, flips e transformações afins, podem alterar significativamente a aparência original das imagens e, em alguns casos, comprometer características visuais essenciais relacionadas às classes de interesse [Cubuk et al. 2020]. Em cenários com poucas amostras (few-shot learning), essa limitação se torna ainda mais crítica, pois a quantidade reduzida de dados reais limita a capacidade do modelo de aprender representações robustas e generalizáveis.

Recentemente, modelos de difusão [Ho et al. 2020, Song et al. 2021b, Dhariwal and Nichol 2021] têm demonstrando alta capacidade de sintetizar imagens fotorrealísticas e semanticamente coerentes. A Stable Diffusion [Rombach et al. 2022] tornou-se uma ferramenta amplamente adotada devido à sua eficiência computacional ao operar no espaço latente comprimido de um Variational Autoencoder (VAE). Dessa forma, surge a hipótese se os modelos de difusão podem servir como mecanismos de

aumentação de dados mais eficazes para o aprendizado auto-supervisionado em cenários com poucas amostras.

Este trabalho explora essa questão através de um estudo comparativo. Nossos objetivos específicos são:

1. Estudar fundamentos teóricos e práticos dos modelos de difusão estável, com ênfase em sua aplicabilidade para geração de dados sintéticos em cenários few-shot.
2. Implementar fine-tuning de modelo pré-treinado (Stable Diffusion) usando LoRA (Low-Rank Adaptation) [Hu et al. 2022]
3. Implementar treinamento do zero de VAE + modelo de difusão latente customizado para o domínio específico.
4. Implementar o DGAE : Autoencoder guiado por difusão para aprendizado latente eficiente.
5. Quantificar o impacto da aumentação por difusão através de métricas de qualidade de agrupamento, comparando cenários com e sem dados sintético, usando BYOL, DGAE [Liu et al. 2025] e CNN-JEPA [Kalapos and Gyires-Tóth 2025]

Utilizamos o dataset Corel, um benchmark clássico com seis categorias visuais distintas (British Guards, Locomotives, Desserts, Salads, Snow Scenes, Sunsets), cada uma contendo aproximadamente 60 imagens.

A estrutura deste relatório segue a seguinte organização: Seção 2 apresenta a motivação e revisão de técnicas; Seção 3 descreve metodologia completa; Seções 4 a 6 apresentarão resultados, discussão e conclusões com base nos experimentos executados.

2. Motivação

2.1. Aprendizado Auto-Supervisionado

O aprendizado auto-supervisionado busca extrair representações úteis de dados não rotulados através da definição de tarefas pretexto que fornecem supervisão [Chen et al. 2021]. No contexto de visão computacional, as abordagens mais bem-sucedidas baseiam-se em dois paradigmas principais:

2.1.1. Aprendizado Contrastivo

SimCLR (Simple Framework for Contrastive Learning of Visual Representations) [Chen et al. 2020]: Uma das primeiras técnicas consolidadas a usar aprendizado contrastivo, define que dado um lote de N imagens, cada uma é aumentada de duas formas diferentes, criando $2N$ visões. O modelo aprende a maximizar a concordância entre aumentações da mesma imagem (pares positivos) enquanto minimiza a similaridade com outras imagens (pares negativos). A função de perda contrastiva normalizada por temperatura ($NT-Xent$) é dada por:

$$\mathcal{L}_{\text{SimCLR}} = -\log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}{\sum_{k=1}^{2N} 1_{[k \neq i]} \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)}$$

onde $\mathbf{z}_i$ e $\mathbf{z}_j$ são embeddings de duas aumentações da mesma imagem, $\text{sim}()$ é similaridade cosseno e τ é a temperatura.

DINO (Self-Distillation with No Labels) [Caron et al. 2021]: Estende o SimCLR através de *self-distillation* com redes teacher-student, onde o professor é atualizado via média exponencial móvel (EMA) dos parâmetros do aluno. DINO-MultiCrop adiciona múltiplas resoluções de crops para enriquecer a supervisão.

2.1.2. Aprendizado Não-Contrastivo

BYOL (Bootstrap Your Own Latent)[Grill et al. 2020]: A principal contribuição foi demonstrar que exemplos negativos não são necessários. A arquitetura consiste em:

- Rede online (treinável): encoder f_θ + projetor g_θ + preditor q_θ
- Rede target (EMA): encoder f_ξ + projetor g_ξ

Dadas duas aumentações v e v' de uma imagem, a perda é:

$$\mathcal{L}_{\text{BYOL}} = \left\| \frac{q_\theta(g_\theta(f_\theta(v)))}{\|q_\theta(g_\theta(f_\theta(v)))\|_2} - \frac{g_\xi(f_\xi(v'))}{\|g_\xi(f_\xi(v'))\|_2} \right\|_2^2$$

com atualização EMA $\xi \leftarrow \tau\xi + (1 - \tau)\theta$, onde $\tau \in [0.99, 0.999]$

CNN-JEPA [Kalapos and Gyires-Tóth 2025]: Propõe uma abordagem baseada em predição de embeddings ao invés de contrastes. A arquitetura utiliza:

- Context encoder: processa a imagem completa excluindo regiões mascaradas
- Target encoder: processa a imagem completa (EMA do context encoder)
- Predictor: prediz embeddings das regiões mascaradas

A perda é simplesmente o erro quadrático médio normalizado entre predição e target nas regiões mascaradas, sem necessidade de aumentações fortes:

$$\mathcal{L}_{\text{JEPA}} = \frac{1}{|M|} \sum_{i \in M} \left\| \frac{p_\theta(h_i^{\text{ctx}})}{\|p_\theta(h_i^{\text{ctx}})\|_2} - \frac{h_i^{\text{tgt}}}{\|h_i^{\text{tgt}}\|_2} \right\|_2^2$$

onde M é o conjunto de posições mascaradas, h_i^{ctx} são embeddings contextuais e h_i^{tgt} são embeddings target.

2.2. Modelos de Stable Diffusion

Todas as técnicas descritas anteriormente dependem fundamentalmente de aumentações de dados. No entanto, aumentações tradicionais possuem limitações, como: (i) alteração semântica, dado que rotações extremas ou mudanças de cor podem alterar a classe percebida; (ii) diversidade limitada, já que não criam novos conteúdos; e baixa generalização, que é crítico para few-shot learning.

Nesse sentido, **Modelos de difusão** [Ho et al. 2020] oferecem uma abordagem diferente que supera as fraquezas da aumento de dados, pois sintetizam novas amostras condicionadas.

2.2.1. Modelos de Difusão Probabilísticos (DDPMs)

Um Modelo de Difusão Probabilístico [Ho et al. 2020] opera através de dois processos complementares:

- Processo direto (forward): Gradualmente corrompe dados reais $\mathbf{x}_0$ adicionando ruído Gaussiano até obter ruído puro $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- Processo reverso (reverse): Aprende a remover ruído iterativamente, gerando novos dados partindo de $\mathbf{x}_T$

A ideia central é que, se conseguirmos reverter um processo de degradação gradual e controlado, podemos transformar ruído aleatório em amostras realistas. Isto contrasta com GANs (que mapeiam ruído diretamente para dados) e VAEs (que aprendem distribuições latentes compactas).

Processo de Difusão Direto

Dado um dado inicial $\mathbf{x}_0 \sim q(\mathbf{x}_0)$, definimos uma cadeia de Markov que adiciona ruído ao longo de T passos:

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I})$$

onde $\{\beta_t\}_{t=1}^T$ é um schedule de ruído com $\beta_t \in (0, 1)$. Dessa forma, β_t pequeno preserva informação e β_t grande degrada rapidamente. Com escolha adequada de β_t , após T passos temos $\mathbf{x}_T \approx \mathcal{N}(\mathbf{0}, \mathbf{I})$.

Podemos amostrar $\mathbf{x}_t$ diretamente de $\mathbf{x}_0$, sem simular a cadeia completa:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

Esta reparametrização acelera o treinamento pois permite gerar $\mathbf{x}_t$ instantaneamente para qualquer t , sem percorrer $t - 1$ passos intermediários.

Processo de Difusão Reverso

A geração consiste em reverter o processo direto:

$$p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t))$$

Objetivo de Treinamento

Otimizar uma perda simplificada de predição de ruído [Ho et al. 2020]:

$$\mathcal{L} = \mathbb{E}_{t \sim \mathcal{U}(1, T), \mathbf{x}_0, \boldsymbol{\epsilon}} [\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\|_2^2]$$

2.2.2. Stable Diffusion: Difusão no Espaço Latente

Embora DDPMs gerem imagens de alta qualidade, operar diretamente no espaço de pixels é computacionalmente custoso. Stable Diffusion resolve isto através de Latent Diffusion Models (LDMs), que realiza o processo de difusão em um espaço latente comprimido aprendido por um VAE.

A arquitetura geral do Stable Diffusion pode ser visualizada abaixo:

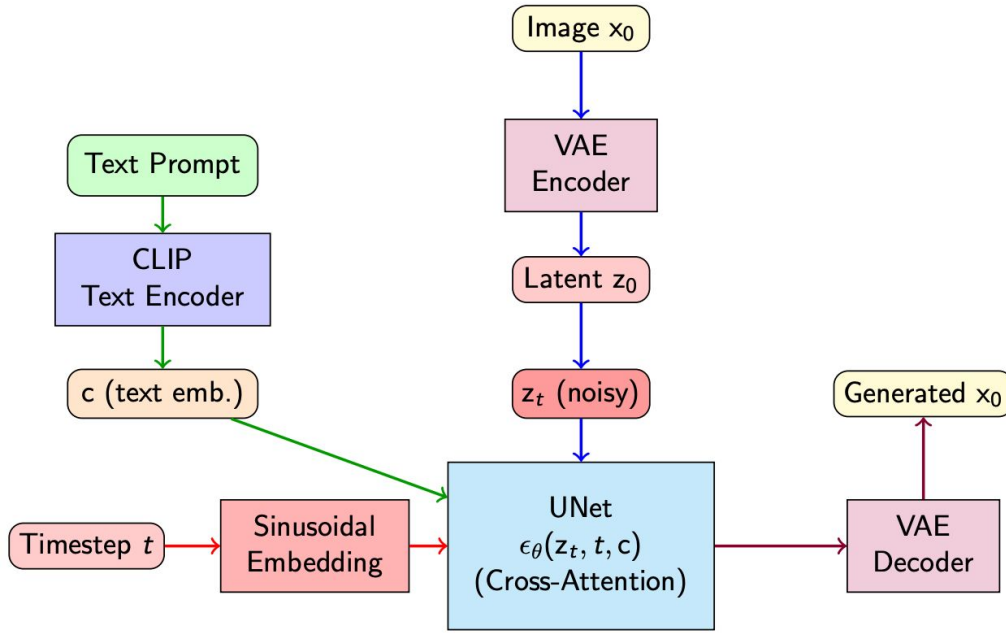


Figure 1. Arquitetura Geral Stable Diffusion

Variational Autoencoder (VAE)

O VAE [Kingma and Welling 2014] consiste em encoder probabilístico e decoder determinístico:

Encoder: $q_{\phi}(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}_{\phi}(\mathbf{x}), \text{diag}(\boldsymbol{\sigma}_{\phi}^2(\mathbf{x})))$

Redes $\boldsymbol{\mu}_{\phi}$ e $\boldsymbol{\sigma}_{\phi}$ (geralmente CNNs) parametrizam distribuição latente. Durante treinamento, amostramos $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \cdot \boldsymbol{\epsilon}$ com $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

Decoder: $p_{\psi}(\mathbf{x}|\mathbf{z})$

Rede generativa que reconstrói $\hat{\mathbf{x}}$ a partir de $\mathbf{z}$.

Objetivo:

$$\mathcal{L}_{\text{VAE}} = \underbrace{\mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})}[-\log p_{\psi}(\mathbf{x}|\mathbf{z})]}_{\text{Reconstruction loss}} + \underbrace{D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x})||p(\mathbf{z}))}_{\text{KL divergence}}$$

onde $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ é o prior. O termo KL regulariza o espaço latente para ser Gaussiano, permitindo sampling de $\mathbf{z}$ durante geração.

Entretanto, a reconstrução MSE pura resulta em imagens borradas. Dessa forma, Stable Diffusion adiciona perda perceptual [Johnson et al. 2016]:

$$\mathcal{L}_{\text{percep}} = \sum_{\ell} \|\phi_{\ell}(\mathbf{x}) - \phi_{\ell}(\hat{\mathbf{x}})\|_1$$

onde ϕ_{ℓ} são features de VGG-16 pré-treinada na camada ℓ . Isto força o decoder a preservar estruturas visuais de alto nível (texturas, bordas), melhorando nitidez.

U-Net Condicionado

O modelo de ruído $\epsilon_{\theta}(\mathbf{z}_t, t, \mathbf{c})$ é uma U-Net [Ronneberger et al. 2015] com arquitetura convolucional com skip connections que preservam informação espacial em diferentes escalas.

Cross-attention: Dado embedding de texto $\mathbf{c} \in \mathbb{R}^{L \times d_c}$ (do CLIP text encoder), aplicamos **cross-attention** em cada camada da U-Net:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^{\top}}{\sqrt{d}}\right) \mathbf{V}$$

onde $\mathbf{Q}$ vem das features espaciais da U-Net $\mathbf{Q} = W_Q \mathbf{F}_{\text{spatial}}$ e $\mathbf{K}, \mathbf{V}$ vêm do texto $\mathbf{K} = W_K \mathbf{c}, \mathbf{V} = W_V \mathbf{c}$

Isto permite que cada região espacial atenda às palavras relevantes do prompt.

Self-attention: permite pixels distantes interagirem (captura dependências globais), enquanto cross-attention injeta informação textual.

2.2.3. Adaptação Few-Shot via LoRA

Fine-tuning completo do Stable Diffusion em poucas amostras enfrenta desafios críticos, como overfitting e custo computacional muito alto.

LoRA (Low-Rank Adaptation) [Hu et al. 2022] oferece uma alternativa ao adicionar camadas adaptadoras de rank baixo que aprendem deltas específicos do domínio-alvo, mantendo modelo base congelado.

Para uma camada linear $\mathbf{h} = W\mathbf{x}$ com $W \in \mathbb{R}^{d \times k}$, a camada LoRA se torna:

$$\mathbf{h} = W\mathbf{x} + B\mathbf{A}\mathbf{x}$$

onde

- $B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times k}$ com $\text{rank } r \ll \min(d, k)$
- W permanece congelado
- Apenas A, B são treinados

Qualquer matriz $\Delta W \in \mathbb{R}^{d \times k}$ pode ser aproximada por BA onde r controla a qualidade da aproximação. Rank pequeno ($r = 4$ - $r = 16$) é suficiente para adaptação de domínio preservando estruturas aprendidas.

LoRA é aplicado nas matrizes de atenção do U-Net, reduzindo drasticamente o espaço treinável e permitindo o treinamento em GPUs mais simples.

2.3. DGAE

Enquanto a abordagem anterior usa difusão para gerar imagens sintéticas que servem como augmentações, o DGAE [Liu et al. 2025] propõe usar um modelo de difusão pré-treinado como guia para o treinamento de um autoencoder convolucional. A ideia central do DGAE consiste em: (i) um encoder E que mapeia a imagem $\mathbf{x}$ para o espaço latente $\mathbf{z}$; (ii) um decoder D que reconstrói $\hat{\mathbf{x}} = D(\mathbf{z})$; (iii) e uma perda de difusão adicional que força $\mathbf{z}$ a estar próximo de latentes gerados pelo modelo de difusão.

A perda do DGAE fica:

$$\mathcal{L}_{\text{DGAE}} = \mathcal{L}_{\text{recon}}(\mathbf{x}, \hat{\mathbf{x}}) + \lambda_{\text{diff}} \mathcal{L}_{\text{diff}}(\mathbf{z}, \mathbf{z}_{\text{diff}})$$

onde $\mathbf{z}_{\text{diff}}$ é obtido passando $\mathbf{x}$ pelo encoder do modelo de difusão. Isso permite aprender representações latentes informadas por prior generativo rico, sem necessidade de augmentação explícita durante o treinamento do autoencoder.

3. Metodologia

3.1. COREL Dataset

Utilizamos o Corel Dataset, um benchmark clássico de recuperação de imagens baseada em conteúdo. O subset selecionado contém 6 classes semanticamente distintas, totalizando 355 imagens com 60 amostras por classe em média. As classes consistem em objetos (guards, trains), alimentos (desserts, salads) e paisagens (snow, sunset).



Figure 2. Exemplos de imagens do COREL Dataset

Pré-processamento: Todas as imagens foram redimensionadas para resolução 256×256 pixels através de *resize* seguido de *center crop*. A normalização aplicada mapeou os valores de pixel de $[0, 255]$ para $[-1, 1]$ usando a transformação padrão

$(\text{pixel}/255 - 0.5)/0.5$. Para os métodos auto-supervisionados (BYOL e CNN-JEPA), aplicamos augmentações adicionais durante o treinamento, incluindo *flip* horizontal ($p = 0.5$) e *color jittering* (brightness, contrast, saturation com intensidade 0.2).

Metadados para Difusão: Criamos um arquivo `metadata.jsonl` mapeando cada imagem do dataset a um prompt textual descritivo de sua classe (Tabela 1). Estes prompts foram projetados para serem informativos sem introduzir ambiguidade, incluindo tanto a descrição semântica quanto qualificadores de qualidade para guiar o modelo de difusão.

3.2. Fine-Tuning LoRA

3.2.1. Arquitetura e Configuração

Utilizamos o Stable Diffusion v1.5 (`runwayml/stable-diffusion-v1-5`) como modelo base, aplicando fine-tuning através de Low-Rank Adaptation (LoRA) para adaptar o modelo ao domínio Corel.

Configuração LoRA:

- **U-Net:** rank $r = 4$, $\alpha = 4$, aplicado nas camadas de atenção
- **Text Encoder:** rank $r = 4$, $\alpha = 4$, aplicado nas projeções
- **Inicialização:** Gaussiana

3.2.2. Treinamento

O treinamento seguiu o protocolo padrão de Latent Diffusion Models, operando no espaço latente do VAE pré-treinado:

- Épocas: 30
- Batch size: 16
- Learning rate: 2×10^{-4} (U-Net), 5×10^{-5} (text encoder)
- Otimizador: AdamW 8-bit ($\beta_1 = 0.9$, $\beta_2 = 0.999$, weight decay= 10^{-2})
- Scheduler: Constante

3.2.3. Geração de Imagens Sintéticas

Após o treinamento, utilizamos o modelo LoRA para gerar 50 imagens sintéticas por classe (300 total), utilizando:

- Euler Discrete scheduler;
- guidance scale 7.5;
- 50 inference steps (DDPM);

As imagens foram geradas em lotes de 1 para garantir diversidade através de diferentes seeds

3.3. Stable Diffusion from Scratch

Para comparação com a abordagem de transfer learning via LoRA, implementamos um pipeline completo de treinamento de Latent Diffusion Model do zero, seguindo a arquitetura proposta por [Rombach et al. 2022].

3.3.1. Variational Autoencoder (VAE)

Arquitetura: Implementamos um VAE convolucional com arquitetura encoder-decoder simétrica. O encoder possui 4 blocos de downsampling (stride 2), reduzindo espacialmente de 256×256 para 16×16 , com canais progressivos [64, 128, 256, 512]. Cada bloco contém Conv2d 4×4 stride 2, BatchNorm2d, LeakyReLU(0.2), e bloco residual ($2 \times$ Conv2d 3×3). O espaço latente possui dimensão 128, com projeções separadas para μ e $\log \sigma^2$. O decoder possui simetria do encoder usando PixelShuffle para upsampling ($2 \times$), preservando qualidade visual.

Objetivo:

$$\mathcal{L}_{\text{VAE}} = \underbrace{\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}[-\log p_\psi(\mathbf{x}|\mathbf{z})]}_{\text{Reconstruction loss}} + \underbrace{D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x})||p(\mathbf{z}))}_{\text{KL divergence}}$$

onde o termo de reconstrução é MSE pixel-wise; KL warmup com β crescendo linearmente de 0 a 1.0 ao longo de 100 épocas; e perda perceptual usa features VGG16 (chamadas até relu3_3) com peso $\lambda_{\text{percep}} = 0.03$.

Treinamento:

- 200 épocas;
- batch size 16;
- learning rate 1×10^{-4}
- otimizador AdamW (weight decay= 1×10^{-5});
- gradient clipping 1.0;
- early stopping (patience=30)
- target KL 25.0

3.3.2. Modelo de Difusão Latente

Arquitetura: Implementamos uma U-Net simplificada operando diretamente em vetores latentes (128-dim) ao invés de mapas de features espaciais. A arquitetura inclui: time embedding sinusoidal (256-dim) $\rightarrow$ MLP; encoder com 3 blocos residuais com dimensões progressivas [256, 512, 512]; bottleneck com 2 blocos residuais (512-dim); decoder com 3 blocos residuais simétricos com skip connections; e output com projeção para dimensão latente (128).

Cada bloco residual contém: Linear $\rightarrow$ GroupNorm $\rightarrow$ SiLU $\rightarrow$ Dropout(0.1) $\rightarrow$ Linear; time embedding injection via projeção aditiva; e skip connections preservando gradientes.

Schedule de difusão: Implementamos dois schedules: (i) Linear com β_t linearmente de 0.0001 a 0.02; e (ii) Cosine com $\bar{\alpha}_t$ seguindo schedule cosseno, mais suave nos extremos. Ambos convergiram satisfatoriamente, com cosine apresentando qualidade ligeiramente superior em timesteps iniciais.

Treinamento:

- 200 épocas;
- batch size 16;

- learning rate 1×10^{-4}
- scheduler CosineAnnealingWarmRestarts ($T_0 = 10$, $T_{\text{mult}} = 2$);
- EMA (decay=0.995);
- early stopping (patience=30)

Objetivo: Predição de ruído no espaço latente:

$$\mathcal{L}_{\text{diff}} = E_{t, z_0, \epsilon} [\|\epsilon - \epsilon_{\theta}(z_t, t)\|^2]$$

Durante cada iteração: imagem $x \rightarrow$ encoder VAE (frozen) $\rightarrow z_0$; sample $t \sim \mathcal{U}(1, 1000)$; $z_t = \sqrt{\bar{\alpha}_t} \cdot z_0 + \sqrt{1 - \bar{\alpha}_t} \cdot \epsilon$; predizer $\hat{\epsilon} = \epsilon_{\theta}(z_t, t)$; e minimizar $\|\epsilon - \hat{\epsilon}\|^2$.

3.3.3. Geração de Imagens

A geração utilizou DDIM sampling [Song et al. 2021a] com 50 steps para acelerar a inferência:

$$\begin{aligned} z_T &\sim \mathcal{N}(0, \mathbf{I}) \\ \text{para } t &\text{ in reverse(timesteps):} \\ \hat{\epsilon} &= \text{model}(z_t, t) \\ z_{0, \text{pred}} &= (z_t - \sqrt{1 - \bar{\alpha}_t} \cdot \hat{\epsilon}) / \sqrt{\bar{\alpha}_t} \\ z_{t-1} &= \sqrt{\bar{\alpha}_{t-1}} \cdot z_{0, \text{pred}} + \sqrt{1 - \bar{\alpha}_{t-1}} \cdot \hat{\epsilon} \\ x &= \text{decoder}(z_0) \end{aligned}$$

Geramos 50 imagens por classe com seeds controladas para reprodutibilidade.

3.4. DGAE

Arquitetura: Encoder-decoder convolucional idêntico ao VAE (seção 3.3.1), mas sem componente probabilístico: encoder $x \rightarrow z$ (determinístico, 128-dim) e decoder $z \rightarrow \hat{x}$.

Extração de features de difusão: Dada uma imagem x , o VAE do Stable Diffusion retorna uma distribuição latente. Extraímos tanto a média μ quanto o desvio padrão σ desta distribuição, concatenamos ao longo da dimensão de canais, e aplicamos global average pooling espacial, resultando em um vetor de features de dimensão 8.

Objetivo:

$$\mathcal{L}_{\text{DGAE}} = \mathcal{L}_{\text{recon}}(\mathbf{x}, \hat{\mathbf{x}}) + \lambda_{\text{diff}} \mathcal{L}_{\text{diff}}(\mathbf{z}, \mathbf{z}_{\text{diff}})$$

Como z (128-dim) e z_{diff} (8-dim) têm dimensionalidades diferentes, aplicamos: (i) normalização L2 em ambos; (ii) projeção de $z_{\text{dgae}} \rightarrow$ primeiras 8 dimensões; e (iii) MSE entre representações normalizadas.

Hiperparâmetros:

- $\lambda = 0.1$;
- 200 épocas;
- batch size 16;
- learning rate 1×10^{-4}

3.4.1. Extração de Features

Após treinamento, extraímos features latentes z para todas as imagens Corel usando o encoder treinado. Estas features (128-dim) foram utilizadas diretamente para clustering, representando um espaço latente informado tanto por reconstrução quanto por priors de difusão.

3.5. BYOL

Implementamos BYOL como baseline de aprendizado auto-supervisionado.

3.5.1. Arquitetura:

Redes duais:

- **Online network:** Encoder $f_\theta \rightarrow$ Projector $g_\theta \rightarrow$ Predictor q_θ (treináveis)
- **Target network:** Encoder $f_\xi \rightarrow$ Projector g_ξ (EMA, não treináveis)

Encoder: CNN com 4 blocos convolucionais $[32 \rightarrow 64 \rightarrow 128 \rightarrow 256]$, stride 2, reduzindo espacialmente até 16×16 .

Projector: Flatten $\rightarrow$ Linear(feature_dim, 512) $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Linear(512, 128).

Predictor: Linear(128, 64) $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Linear(64, 128).

3.5.2. Treinamento:

Augmentações BYOL: Criamos pares de views através de: rotação aleatória (± 20); transformação afim (translate 10%, scale 0.9-1.1); flip horizontal ($p = 0.5$); color jitter (brightness/contrast/saturation ± 0.2); e ruído Gaussiano ($\sigma = 0.05$).

Objetivo:

$$\mathcal{L}_{\text{BYOL}} = \left\| \frac{q_\theta(g_\theta(f_\theta(v)))}{\|q_\theta(g_\theta(f_\theta(v)))\|_2} - \frac{g_\xi(f_\xi(v'))}{\|g_\xi(f_\xi(v'))\|_2} \right\|_2^2$$

Minimizamos a distância entre predições online e projeções target para ambas as views.

Atualização EMA: $\xi \leftarrow \tau \cdot \xi + (1 - \tau) \cdot \theta$ onde τ cresce linearmente de 0.99 a 0.999 ao longo das épocas.

Hiperparâmetros:

- 200 épocas;
- batch size 16
- learning rate 3×10^{-3}
- scheduler StepLR (step_size=50, gamma=0.5)
- projection dim 128

Extração de features: Após treinamento, usamos apenas o encoder online f_θ , extraíndo features antes do flatten (shape: batch $\times$ 256 $\times$ 16 $\times$ 16), que são então achatadas para vetor 1D.

3.6. CNN-JEPA

3.6.1. Arquitetura

Context Encoder: ResNet-like com 4 layers: Conv 7×7 stride 2 $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ MaxPool; Layer1 com $2 \times$ ResBlock(64, 64); Layer2 com $2 \times$ ResBlock(64, 128, stride 2); Layer3 com $2 \times$ ResBlock(128, 256, stride 2); Layer4 com $2 \times$ ResBlock(256, 256, stride 2); e AdaptiveAvgPool $\rightarrow$ embedding 256-dim.

Target Encoder: Mesma arquitetura, com parâmetros atualizados via EMA.

Predictor: MLP com 3 camadas $[256 \rightarrow 512 \rightarrow 512 \rightarrow 256]$ com BatchNorm e ReLU.

3.6.2. Masking Strategy

Aplicamos masking espacial para criar contexto e target: máscara retangular aleatória cobrindo 30% da imagem; posição aleatória a cada iteração; context (imagem com região mascarada zerada); e target (imagem completa). Esta estratégia força o modelo a aprender representações que podem prever conteúdo faltante, similar ao masked image modeling.

3.6.3. Treinamento

Objetivo:

$$\mathcal{L}_{\text{JEPA}} = \frac{1}{|M|} \sum_{i \in M} \left\| \frac{p_{\theta}(h_i^{\text{ctx}})}{\|p_{\theta}(h_i^{\text{ctx}})\|_2} - \frac{h_i^{\text{tgt}}}{\|h_i^{\text{tgt}}\|_2} \right\|_2^2$$

onde h^{ctx} e h^{tgt} são embeddings do context e target encoders, respectivamente.

Hiperparâmetros:

- 200 épocas
- batch size 16
- learning rate 1×10^{-3}
- scheduler CosineAnnealingLR
- EMA momentum $0.99 \rightarrow 0.999$

Extração de features: Usamos o context encoder após treinamento, obtendo embeddings 256-dim para cada imagem.

3.7. Avaliação dos Resultados

3.7.1. Cenários

Avaliamos três cenários para cada método de extração de features:

1. **Original:** Features extraídas apenas das 355 imagens reais do Corel
2. **+LoRA:** Features das 355 imagens originais + 300 imagens sintéticas geradas via LoRA (total: 655)
3. **+Diffusion:** Features das 355 originais + 300 sintéticas geradas via VAE+Diffusion do zero (total: 655)

3.7.2. Clustering

Pré-processamento: Aplicamos StandardScaler (média 0, variância 1) às features extraídas para normalizar escalas entre diferentes métodos.

Clustering: K-Means com $k = 6$ (número de classes ground-truth), inicialização k-means++ com $n_{init} = 10$ para estabilidade.

Métricas de avaliação:

1. **Silhouette Score:** Mede coesão intra-cluster vs. separação inter-cluster

$$s = \frac{b - a}{\max(a, b)}$$

Range: $[-1, 1]$, valores maiores indicam melhor clustering.

2. **Adjusted Rand Index (ARI):** Mede concordância entre clustering e ground-truth, ajustado para chance

$$ARI = \frac{RI - E[RI]}{\max(RI) - E[RI]}$$

Range: $[-1, 1]$, 1 = perfect match, 0 = random.

3. **Normalized Mutual Information (NMI):** Informação mútua normalizada entre clusters e labels verdadeiros

$$NMI = \frac{2 \cdot I(U; V)}{H(U) + H(V)}$$

Range: $[0, 1]$, 1 = correspondência perfeita.

3.7.3. Visualizações

Utilizamos UMAP (Uniform Manifold Approximation and Projection) para gerar visualizações separadas para cada combinação método-cenário (9 plots UMAP), utilizando cores fixas por classe para facilitar comparação visual entre cenários.

3.8. Ambiente de Execução e Reprodutibilidade

Hardware: Foi utilizada a plataforma Google Colab com GPU NVIDIA L4 40GB; CPU Intel Xeon; RAM 32GB

Software: Python 3.10+, PyTorch 2.0+, Scikit-learn, UMAP-learn.

Reprodutibilidade: Seeds fixadas (42 padrão em todos os scripts); Torch deterministic mode habilitado quando possível; e logs detalhados de hiperparâmetros com checkpoints salvos a cada 10-50 épocas.

O código-fonte completo pode ser encontrado em <https://github.com/ccavalieri/MO433>.

4. Resultados

As imagens geradas tanto pela estratégia com LoRA quanto pelo modelo de difusão from scratch são apresentadas a seguir.



Figure 3. Exemplos de imagens sintéticas geradas por LoRA

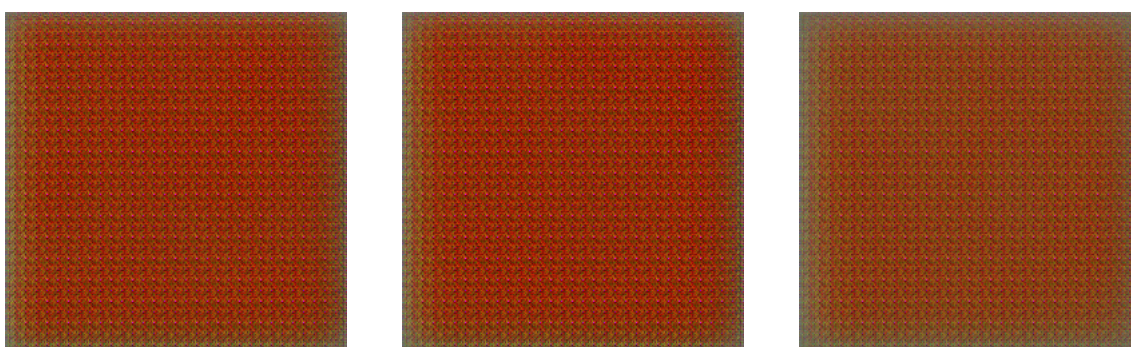


Figure 4. Exemplos de imagens sintéticas geradas por Stable Diffusion from scratch

A seguir, são apresentadas as métricas de agrupamento para cada par método-cenário.

Método	Cenário	Silhouette	ARI	NMI
CNN-JEPA	Original	0.260	0.170	0.233
CNN-JEPA	+LoRA	0.248	0.086	0.129
CNN-JEPA	+Diffusion	0.239	0.090	0.119
BYOL	+LoRA	0.120	0.264	0.325
BYOL	+Diffusion	0.099	0.331	0.422
BYOL	Original	0.089	0.637	0.673
DGAE	+LoRA	0.085	0.149	0.211
DGAE	Original	0.079	0.288	0.437
DGAE	+Diffusion	0.079	0.215	0.302

Por fim, a imagem abaixo apresenta as visualizações UMAP para cada um dos cenários e métodos.

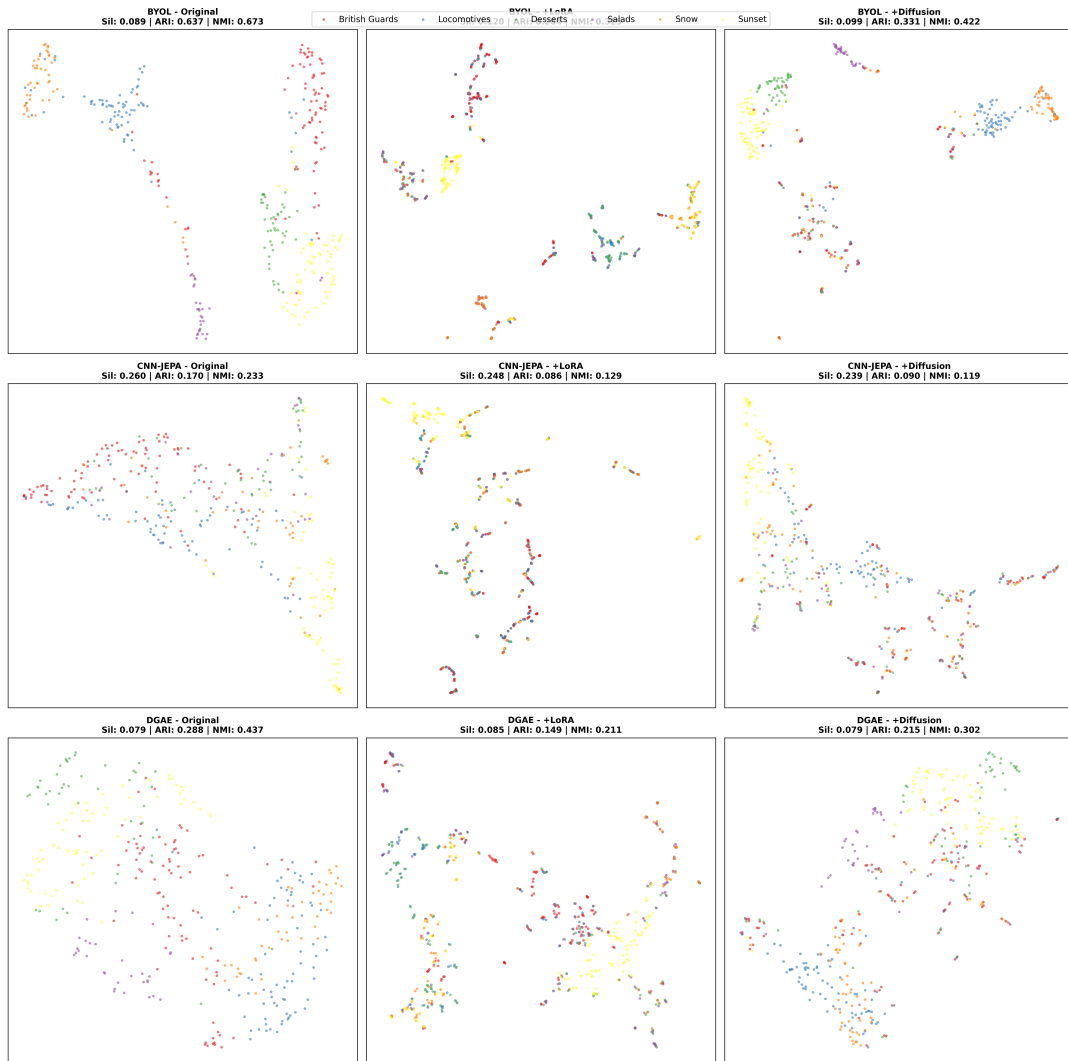


Figure 5. UMAP para todos os cenários e métodos

5. Discussão

Os resultados experimentais revelam que a adição de imagens sintéticas geradas por modelos de difusão degradou sistematicamente o desempenho de agrupamento em todos os três métodos avaliados. Esse comportamento pode indicar uma série de problemas ligados a distribuição das imagens sintéticas geradas, assim como de colapso dos modelos treinados.

5.1. Análise por Método

BYOL:

O BYOL apresentou o melhor desempenho absoluto no cenário Original (ARI=0.637, NMI=0.673), demonstrando que sua estratégia de aprendizado com redes teacher-student conseguiu extrair representações semanticamente coerentes das 355 imagens reais. No entanto, a adição de imagens sintéticas causou degradação severa:

- **+LoRA:** ARI caiu de 0.637 para 0.264 (-58.5%), NMI de 0.673 para 0.325 (-51.7%)

- **+Diffusion:** ARI 0.331 (-48.0%), NMI 0.422 (-37.3%)

Esta degradação sugere que o BYOL é particularmente sensível à qualidade distribucional dos dados. As aumentações fortes aplicadas durante o treinamento BYOL (rotação $\pm 20^\circ$, color jitter, ruído Gaussiano) podem ter criado um espaço de representação que não generaliza bem para as características das imagens sintéticas, que possuem artefatos e distribuições estatísticas distintas das imagens reais.

CNN-JEPA:

O CNN-JEPA apresentou o maior Silhouette Score em todos os cenários (0.260 Original, 0.248 +LoRA, 0.239 +Diffusion), indicando clusters internamente coesos e bem separados. Entretanto, os valores de ARI e NMI foram consistentemente baixos (ARI: 0.170→0.086→0.090), revelando uma discordância entre a estrutura de agrupamento descoberta e as classes ground-truth.

Esta diferença pode ser explicada pela natureza da tarefa de predição de regiões mascaradas do JEPA. Enquanto o modelo aprende representações que capturam estruturas visuais locais (texturas, padrões geométricos), estas características podem não corresponder aos atributos semânticos de alto nível que definem as seis classes do Corel. Por exemplo, o modelo pode agrupar imagens por similaridade de cor/textura (céu azul, vegetação verde) independentemente da categoria semântica (snow vs. sunset vs. guards).

DGAE:

O DGAE apresentou desempenho intermediário (Original: ARI=0.288, NMI=0.437), superando o CNN-JEPA em concordância com ground-truth mas ficando atrás do BYOL. A degradação com imagens sintéticas seguiu o padrão observado:

- **+LoRA:** ARI 0.149 (-48.3%), NMI 0.211 (-51.7%)
- **+Diffusion:** ARI 0.215 (-25.3%), NMI 0.302 (-30.9%)

Interessantemente, o cenário +Diffusion teve degradação menos severa que +LoRA, sugerindo uma possível compatibilidade entre o prior de difusão usado durante o treinamento DGAE (features do VAE do Stable Diffusion) e as características das imagens geradas pelo modelo from scratch.

5.2. Análise da Qualidade das Imagens Sintéticas

As imagens geradas (Figuras 3 e 4) revelam diferenças críticas. No LoRA as imagens apresentam boa qualidade e coerência semântica com as classes originais. Já no Diffusion from Scratch as imagens exibem falha catastrófica de qualidade, apresentando texturas repetitivas sem estrutura semântica reconhecível. Este colapso pode ser atribuído a múltiplos fatores, como colapso posterior no VAE ou Underfitting do Modelo de Difusão

Apesar da boa qualidade visual das imagens LoRA, ambas as estratégias de aumento degradaram o desempenho, sugerindo que o problema não é puramente de qualidade visual, mas sim de compatibilidade distribucional.

5.3. Análise das Visualizações UMAP

BYOL Original: Apresenta separação clara entre classes, com clusters compactos e sobreposição mínima. As classes snow (laranja) e sunset (amarelo) formam regiões distintas, validando os altos valores de ARI/NMI.

BYOL +LoRA e +Diffusion: A estrutura de clusters torna-se difusa, com maior sobreposição entre classes. É bastante visível a fragmentação da classe guards (vermelho), que se dispersa em múltiplos subclusters.

CNN-JEPA (todos cenários): Clusters apresentam alta densidade local (explicando alto Silhouette) mas com fronteiras de decisão que não respeitam as categorias semânticas. Observa-se a formação de superclusters que agrupam múltiplas classes ground-truth, explicando o baixo ARI.

DGAE: Padrão intermediário, com separação moderada entre classes no cenário Original, deteriorando progressivamente com adição de imagens sintéticas.

6. Conclusão

Este trabalho investigou o impacto da aumentação de dados baseada em difusão no aprendizado auto-supervisionado de espaços latentes em cenários few-shot, utilizando o dataset Corel como benchmark. Implementamos e comparamos três métodos de extração de características (BYOL, CNN-JEPA, DGAE) em três cenários (Original, +LoRA, +Diffusion), avaliando qualidade de agrupamento via Silhouette Score, ARI e NMI.

A adição de imagens geradas por difusão (LoRA ou from scratch) reduziu consistentemente as métricas de concordância com ground-truth em todos os métodos avaliados. O BYOL sofreu degradação mais severa (ARI -58% com LoRA), enquanto CNN-JEPA mostrou maior robustez relativa.

Com $ARI=0.637$ e $NMI=0.673$, o BYOL no cenário sem aumentação sintética superou significativamente os demais métodos, demonstrando que aprendizado com apenas dados reais pode extrair representações de alta qualidade mesmo com poucas amostras.

Como principais dificuldades e pontos de melhoria, destacamos a escassez de recursos computacionais para treinamento, que toranaram a execução dos scripts custosa a cada versão implementada. Além disso, a ausência do código-fonte utilizado pelos autores do método DGAE dificultou a reprodução fiel dos resultados do artigo.

Direções promissoras para trabalhos futuros incluem desenvolver objetivos de treinamento que explicitamente minimizem divergência distribucional entre features de amostras reais e sintéticas. Além disso, ao invés de usar todas as imagens geradas, desenvolver critérios de qualidade/diversidade para filtrar apenas as mais benéficas.

References

- Caron, M., Touvron, H., Misra, I., Jégou, H., Mairal, J., Bojanowski, P., and Joulin, A. (2021). Emerging properties in self-supervised vision transformers. In *Proceedings of the International Conference on Computer Vision (ICCV)*.
- Chen, T., Kornblith, S., Norouzi, M., and Hinton, G. (2020). A simple framework for contrastive learning of visual representations. In *Proceedings of the International Conference on Machine Learning (ICML)*.
- Chen, T., Luo, C., and Li, L. (2021). Intriguing properties of contrastive losses. In *Advances in Neural Information Processing Systems (NeurIPS)*.

- Cubuk, E. D., Zoph, B., Shlens, J., and Le, Q. V. (2020). Randaugment: Practical automated data augmentation with a reduced search space. In *Computer Vision and Pattern Recognition Workshops (CVPRW)*.
- Dhariwal, P. and Nichol, A. (2021). Diffusion models beat gans on image synthesis. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Grill, J.-B., Strub, F., Altché, F., Tallec, C., Richemond, P. H., Buchatskaya, E., Doersch, C., Pires, B. A., Guo, Z. D., Azar, M. G., Piot, B., Kavukcuoglu, K., Munos, R., and Valko, M. (2020). Bootstrap your own latent: A new approach to self-supervised learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Ho, J., Jain, A., and Abbeel, P. (2020). Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. (2022). Lora: Low-rank adaptation of large language models. In *Proceedings of the International Conference on Learning Representations (ICLR)*.
- Jing, L. and Tian, Y. (2020). Self-supervised visual feature learning with deep neural networks: A survey. In *IEEE Transactions on Pattern Analysis and Machine Intelligence*, volume 43, pages 4037–4058.
- Johnson, J., Alahi, A., and Fei-Fei, L. (2016). Perceptual losses for real-time style transfer and super-resolution. In *Proceedings of the European Conference on Computer Vision (ECCV)*.
- Kalapos, A. and Gyires-Tóth, B. (2025). Cnn-jepa: Self-supervised pretraining convolutional neural networks using joint embedding predictive architecture.
- Kingma, D. P. and Welling, M. (2014). Auto-encoding variational bayes. In *Proceedings of the International Conference on Learning Representations (ICLR)*.
- LeCun, Y. and Misra, I. (2021). Self-supervised learning: The dark matter of intelligence. Meta AI Blog.
- Liu, D., Peng, Y., Tang, H., Chen, Y., Han, C., Ge, Z., Jiang, D., and Liao, M. (2025). Dgae: Diffusion-guided autoencoder for efficient latent representation learning.
- Rombach, R., Blattmann, A., Lorenz, D., Esser, P., and Ommer, B. (2022). High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Ronneberger, O., Fischer, P., and Brox, T. (2015). U-net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*.
- Song, J., Meng, C., and Ermon, S. (2021a). Denoising diffusion implicit models. In *Proceedings of the International Conference on Learning Representations (ICLR)*.
- Song, Y., Sohl-Dickstein, J., Kingma, D. P., Kumar, A., Ermon, S., and Poole, B. (2021b). Score-based generative modeling through stochastic differential equations. In *Proceedings of the International Conference on Learning Representations (ICLR)*.